Newton School of Technology

19

Lecture 19

Memory Management I: Hierarchy and Contiguous Allocation

by Brijesh Shukla

C07: Operating Systems

Newton School of Technology

2

Join the lecture online on your dashboard

Today’s Agenda

Newton School of Technology

- Introduction to Memory Management
- Memory Hierarchy and Characteristics
- Contiguous Allocation: Fixed Partitions
- Dynamic Partitioning Algorithms
- Fragmentation and Solutions
- Swapping and Overlays

Introduction to Memory Management

Newton School of Technology

🧠 Definition

Memory management is how the **OS controls and optimizes main memory** for running processes.

⚙️ Core Responsibilities

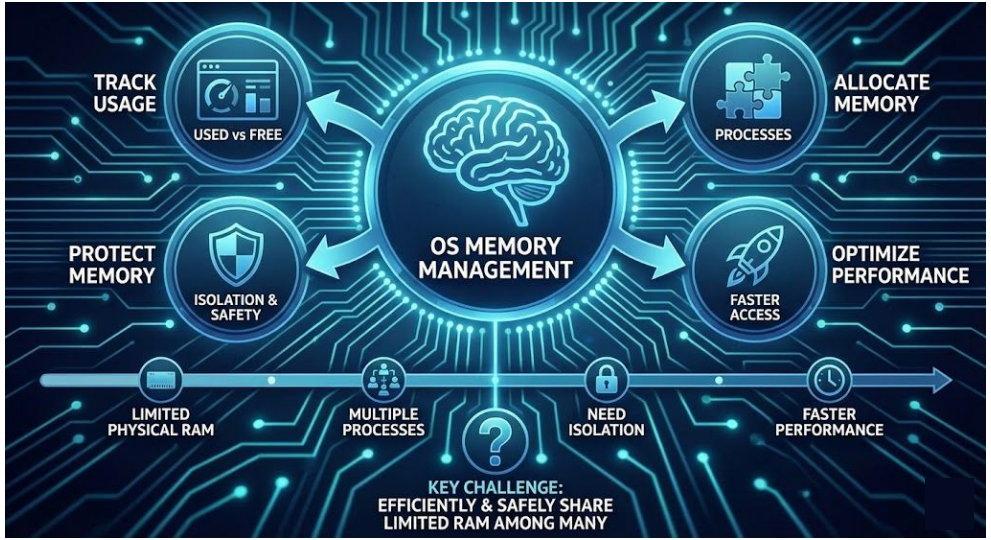
- 📊 Track **used vs free** memory
- ➕ Allocate memory to processes
- ➖ Free memory when done
- 🔐 Protect process memory
- 🚀 Optimize performance

❓ Why It Matters

- 🧱 Physical memory is **limited**
- 👥 Multiple processes compete
- 🔒 Need isolation & safety
- ⏱️ Faster memory = better performance
- ♻️ Efficient use of available RAM

🎯 Key Challenge

How can limited physical memory be shared **efficiently and safely** among many processes?



Objectives of Memory Management

Newton School of Technology

- 🔄 **Relocation**
  Load programs **anywhere in memory**; support dynamic execution
- 🔐 **Protection**
  Isolate processes; **protect OS/kernel** memory
- 🤝 **Sharing**
  Controlled memory sharing (libraries, IPC)
- 🧩 **Logical Organization**
  Support modules, separate compile & link
- 💾 **Physical Organization**
  Move data between RAM ↔ storage, **transparent to user**
- ⚡ **Efficiency**
  Minimize waste, faster access, better CPU use

Key Takeaway
Memory management balances **safety, flexibility, and performance** while using limited RAM efficiently.

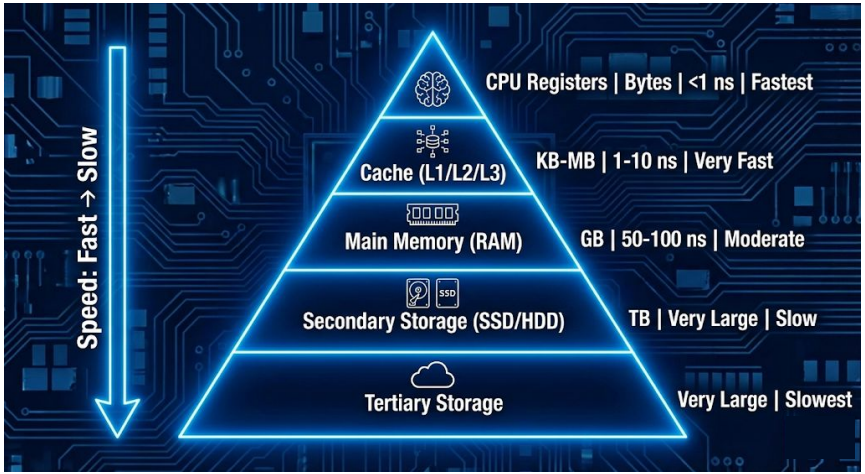
The Memory Hierarchy

Newton School of Technology

🧠 **Core Idea**
Memory is organized in **layers** based on **speed, cost, and capacity**
➡️ **Faster = Smaller & Costlier**
➡️ **Slower = Larger & Cheaper**

🏎️ **Hierarchy (Top → Bottom)**

- ⚡ **CPU Registers**
  Bytes | **< 1 ns** | Fastest, most expensive
- 🚀 **Cache (L1 / L2 / L3)**
  KB–MB | **1–10 ns** | Very fast
- 💾 **Main Memory (RAM)**
  GB | **50–100 ns** | Moderate speed & cost
- 📀 **Secondary Storage (SSD/HDD)**
  TB | **µs–ms** | Slow, cheap
- ☁️ **Tertiary Storage (Tape/Cloud)**
  Very large | **seconds+** | Slowest, cheapest



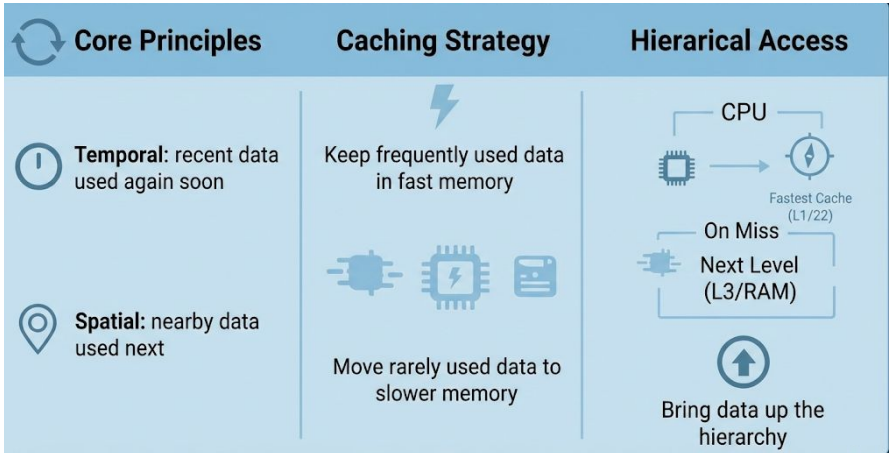
🎯 **Key Takeaway**
Systems trade **speed for size and cost**—fast memory is small, slow memory is large.

Memory Hierarchy Characteristics

Newton School of Technology

📌 Core Principles

- 🔁 **Locality of Reference**
  - **Temporal:** recent data used again soon
  - **Spatial:** nearby data used next
- ⚡ **Caching Strategy**
  - Keep **frequently used** data in fast memory
  - Move **rarely used** data to slower memory
- 🧭 **Hierarchical Access**
  - CPU checks **fastest level first**
  - On miss → check next level
  - Bring data **up the hierarchy**



📊 Performance Impact

- **Hit Rate:** % found in fast memory
- **Effective Access Time:** `Hit Rate × Fast Access + Miss Rate × Slow Access`

Key Takeaway

High locality + good caching = faster memory access and better performance.

Main Memory Organization

Newton School of Technology

🧠 Memory Basics

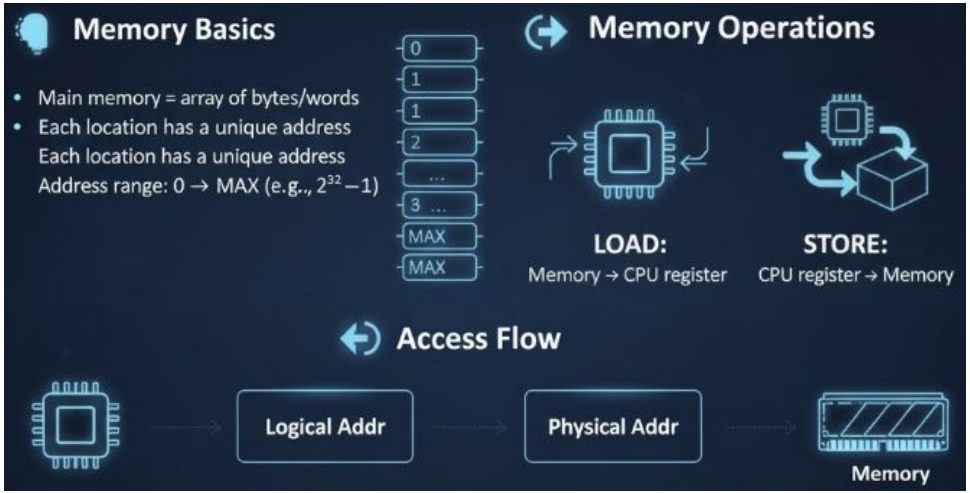
- Main memory = array of bytes/words
- Each location has a **unique address**
- Address range: `0 → MAX` (e.g., $2^{32} - 1$)

🔄 Memory Operations

- **LOAD:** Memory → CPU register
- **STORE:** CPU register → Memory

🧭 Address Spaces

- **Physical Address**
  - Real RAM address
- **Logical / Virtual Address**
  - Generated by CPU/process
  - Translated to physical address



🔁 Access Flow

`CPU → Logical Addr → Translation → Physical Addr → Memory`

Main Memory Organization

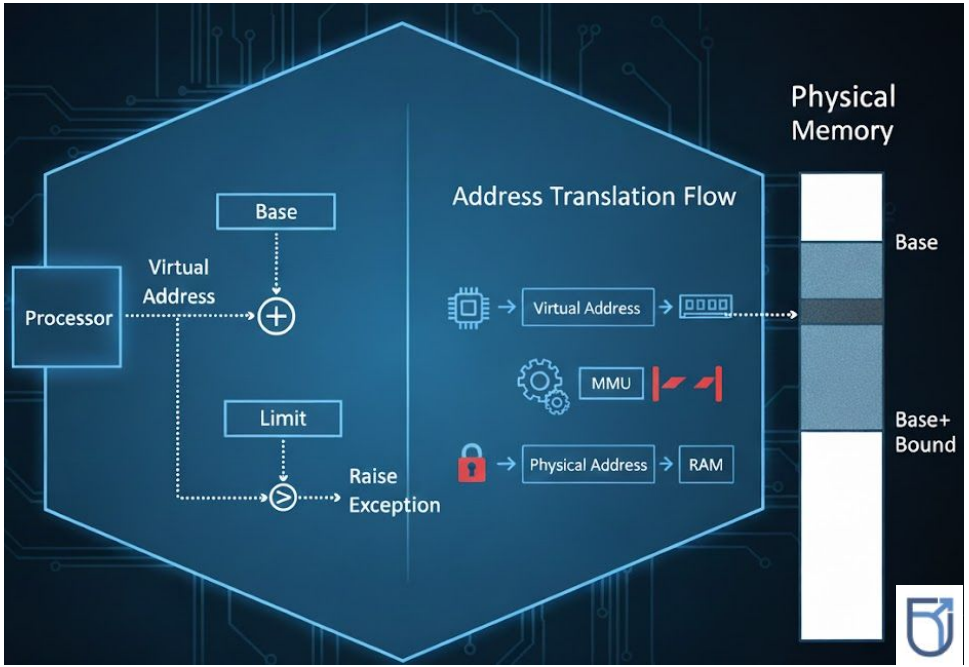
Newton School of Technology

🔐 **Memory Protection**

- **Base Register:** start of process memory
- **Limit Register:** size of process memory
- **Check:** `Base ≤ Address < Base + Limit`

Key Takeaway

Programs use virtual addresses; hardware safely maps them to physical memory.



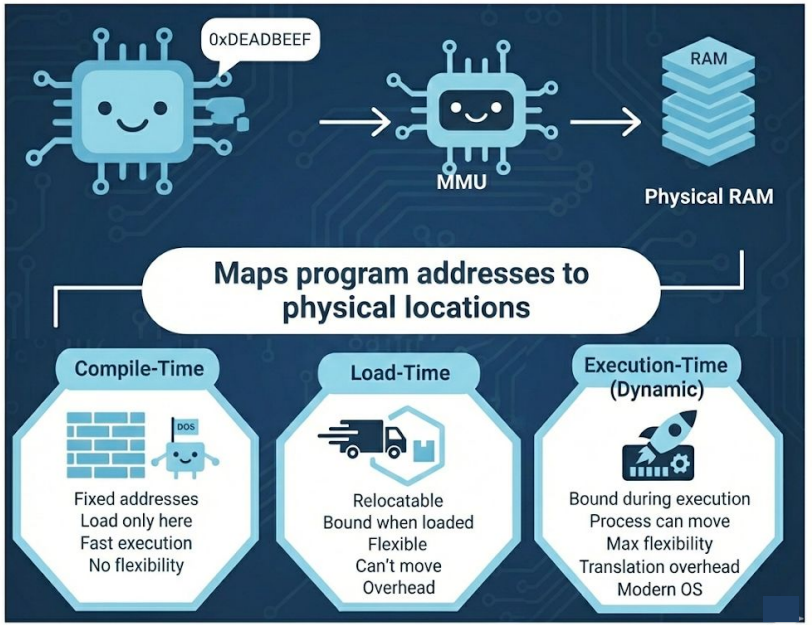
Address Binding

Newton School of Technology

🧩 **What is Address Binding?**
Mapping **program addresses** to **actual memory locations**

🕑 **Types of Binding**

- 🧱 **Compile-Time**
  - Fixed (absolute) addresses
  - Load at one location only
  - ✔ Fast | ❌ No flexibility
  - Example: old DOS programs
- 📦 **Load-Time**
  - Relocatable code
  - Bound when loaded
  - ✔ More flexible | ❌ Can't move while running
- 🔄 **Execution-Time (Dynamic)**
  - Bound during execution
  - Process can move in memory
  - ✔ Maximum flexibility | ❌ Translation overhead
  - Used in modern OS

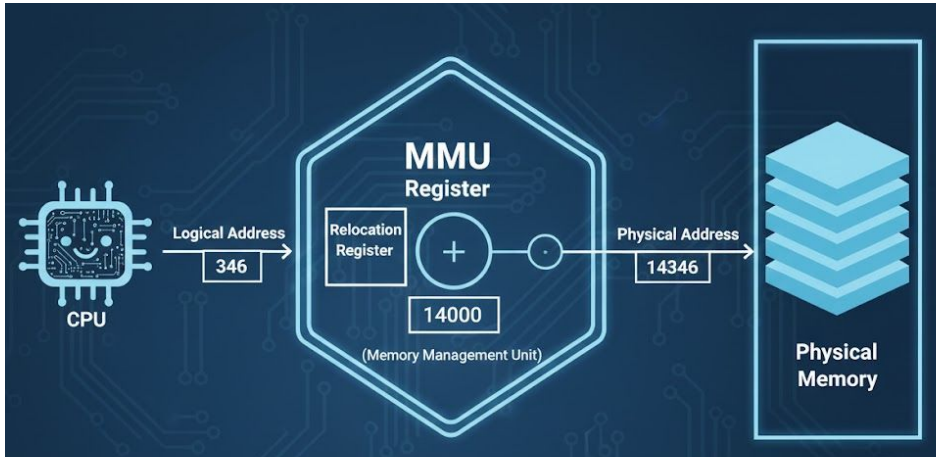


Address Binding

Newton School of Technology

⚙️ **Address Translation**
`Logical Address + Relocation Register → Physical Address`
Handled by **MMU (hardware)**

Key Takeaway
Later binding = more flexibility; modern systems use **execution-time binding** for safety and efficiency.



Protection in Contiguous Allocation

Newton School of Technology

🔐 **Why Protection Is Needed**

- Prevent process → process memory access
- Protect OS from user programs
- Catch invalid memory references
- Ensure stability & security

⚙️ **Hardware Support: Base & Limit Registers**

- **Base Register:** Start address of process
- **Limit Register:** Size of process memory

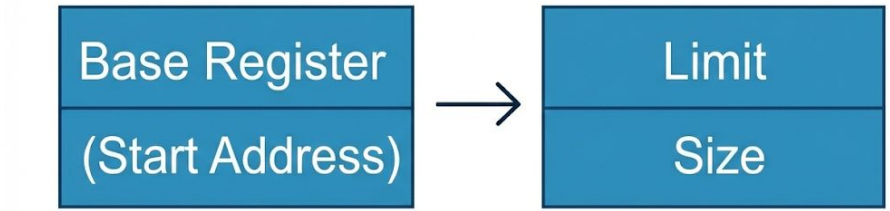
Check on every access:

```
Physical Address = Base + A
```

- If `A < 0` or `A ≥ Limit` → **TRAP**
- Else → **Access allowed**

📊 **Example**

- Base = **300KB**, Limit = **120KB**
- Valid range: **300–420KB**



Physical Address = Base + A

If A < 0 or A ≥ Limit → TRAP

Else → Access allowed

Protection in Contiguous Allocation

Newton School of Technology

🧠 **Where It's Used**

- **Fixed Partitions:**
  - Base = partition start
  - Limit = partition size
- **Dynamic Partitions:**
  - Base & limit set per allocation

🚨 **What Protection Catches**

- Buffer overflows
- Invalid pointers
- Out-of-bounds array access

➡️ OS raises **exception** and terminates process

🧑‍💻 **Privilege Levels**

- **Kernel Mode:**
  - Access all memory
  - Can change base/limit
- **User Mode:**
  - Restricted to its memory only

Key Takeaway

Base & limit registers provide **simple, fast, hardware-level memory protection** in contiguous allocation systems.

Contiguous Memory Allocation

Newton School of Technology

🧠 **Definition**

Each process occupies **one continuous block** of physical memory.

📦 **Core Idea**

- Process uses **consecutive addresses**
- No gaps **inside** a process
- Entire process loaded as **one block**

🗂️ **Simple Memory Layout**

- OS in resident memory
- User processes placed **one after another**
- Remaining space = free memory

🔀 **Two Approaches**

- 🧱 **Fixed Partitioning**
  Predefined, fixed-size partitions
- 📐 **Dynamic Partitioning**
  Partitions created based on process size

✅ **Advantages**

- Simple design, Fast access and Low overhead

❌ **Disadvantages**

- Fragmentation, Poor flexibility and Limits process size

Key Takeaway

Contiguous allocation is fast and simple, but wastes memory and lacks flexibility.

Fixed Partitioning - Concept

Newton School of Technology

🧠 **Definition**
Memory is split into **fixed-size partitions** at system startup.
➡️ **One process per partition**

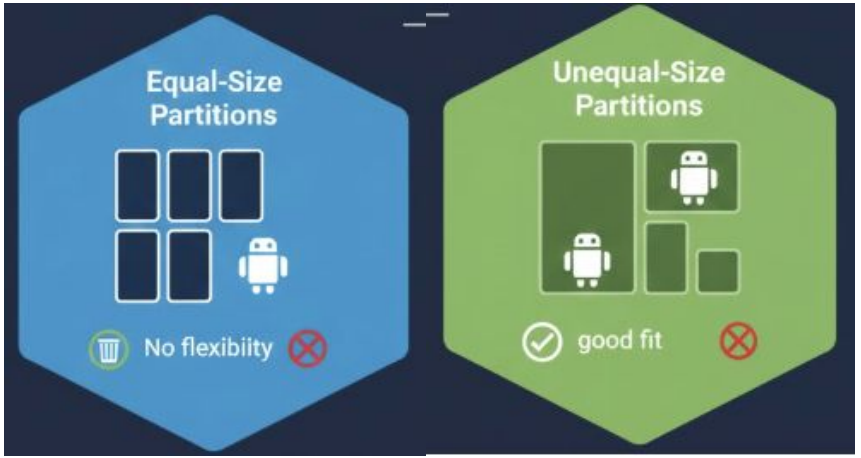
⚙️ **Key Characteristics**

- Partition sizes decided **before boot**
- Sizes **do not change** during execution
- Process must **fit entirely** in a partition

🔀 **Variants**

- 🟦 **Equal-Size Partitions**
  Same size; simple; causes more waste

- 🟩 **Unequal-Size Partitions**
  Different sizes; better fit for varied processes



Fixed Partitioning - Concept

Newton School of Technology

📦 **Example (Equal Size: 5 MB)**

- P1 (3 MB) → **2 MB wasted**
- P2 (1 MB) → **4 MB wasted**
- P3 (4 MB) → **1 MB wasted**

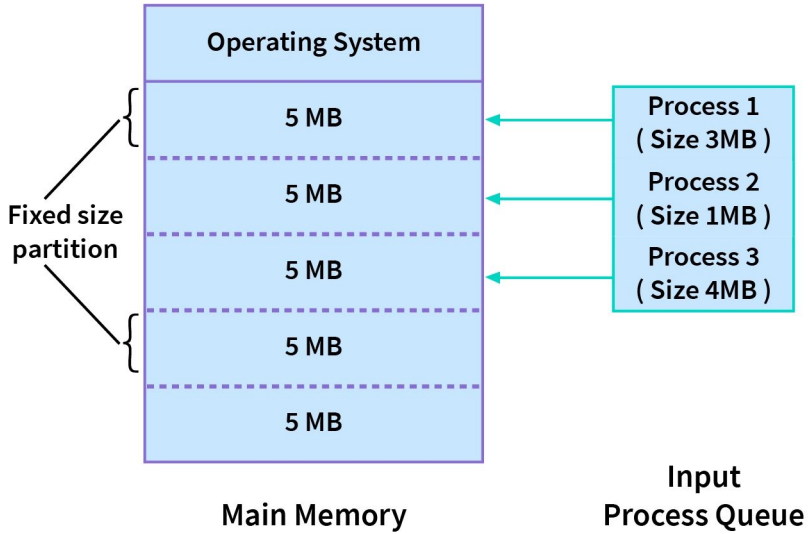
➡️ **Internal Fragmentation**

✅ **Advantages**

- Simple & predictable
- Easy allocation

❌ **Disadvantages**

- Internal fragmentation
- Fixed process limit
- Poor fit for large/variable processes



Key Takeaway

Fixed partitioning is easy to manage but wastes memory due to internal fragmentation.

Fixed Partitioning – Placement Strategies

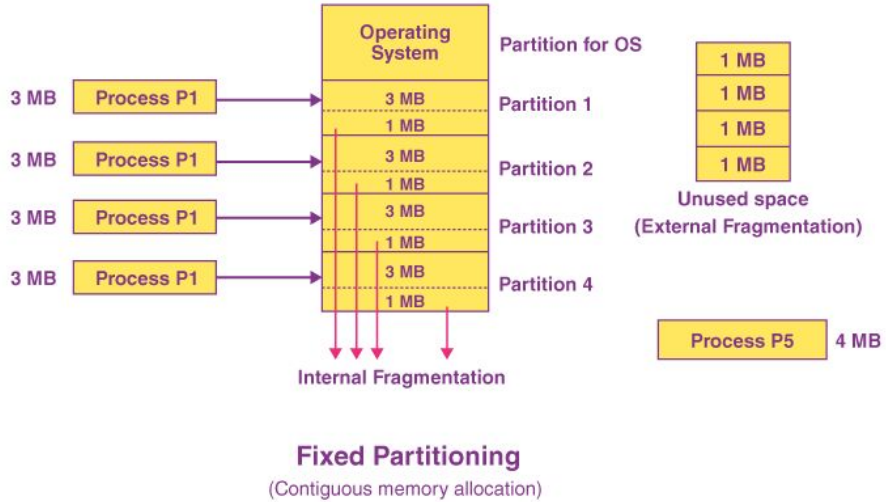
Newton School of Technology

✅ **Best Practices**

- Prefer **unequal partitions**
- Match sizes to workload
- Review & adjust over time

Key Takeaway

Placement choice trades **flexibility** for **simplicity**. Internal fragmentation is unavoidable in fixed partitioning.



Fixed Partitioning

(Contiguous memory allocation)

Dynamic Partitioning – Concept

Newton School of Technology

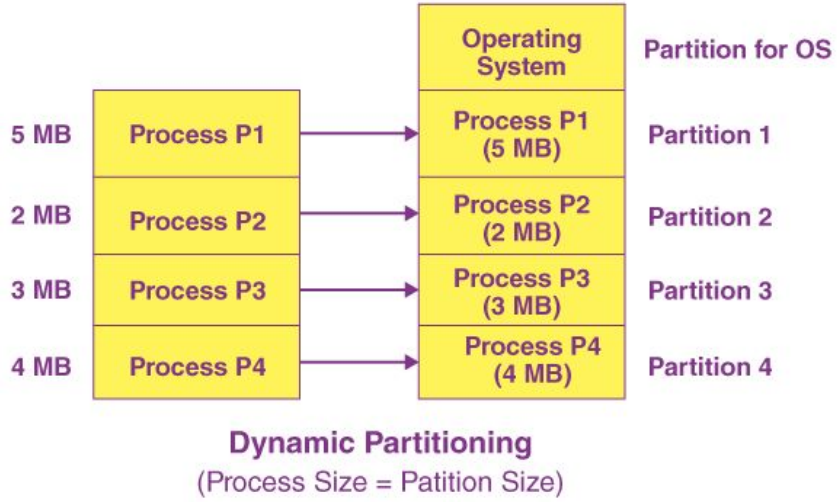
🧠 **Definition**
Memory partitions are **created on demand**, sized **exactly to the process**.

⚙️ **Key Features**

- No fixed partition sizes
- **Partition = process size**
- Created on load, removed on exit
- ❌ No **internal fragmentation**

📊 **How Memory Changes**

- Start: OS + large free block
- Load processes → memory splits dynamically
- Free space shrinks and shifts over time



Dynamic Partitioning
(Process Size = Patition Size)

Dynamic Partitioning – Concept

✅ **Advantages**

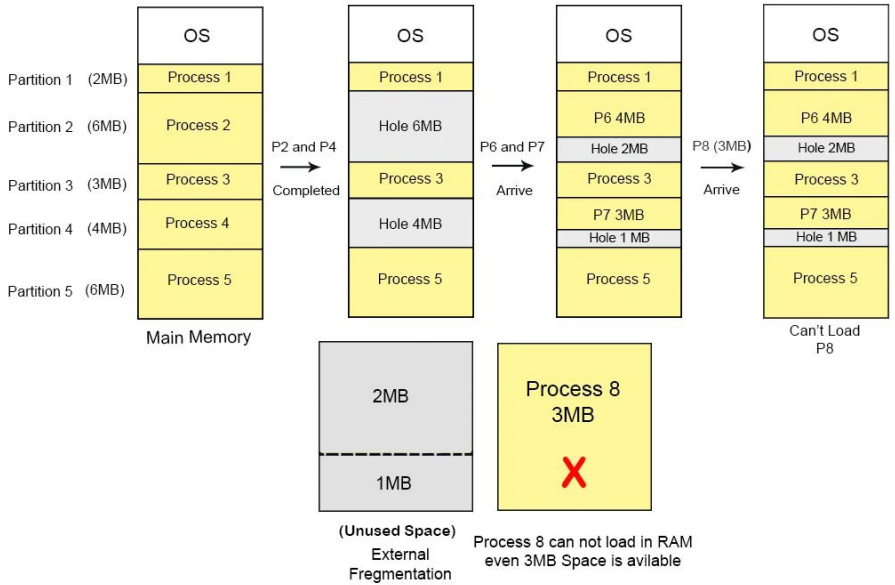
- No internal waste
- Flexible for varied process sizes
- Better initial memory use

❌ **Disadvantages**

- **External fragmentation** develops
- More complex allocation logic
- Overhead tracking free blocks

Key Takeaway

Dynamic partitioning fits processes perfectly but suffers from external fragmentation over time.



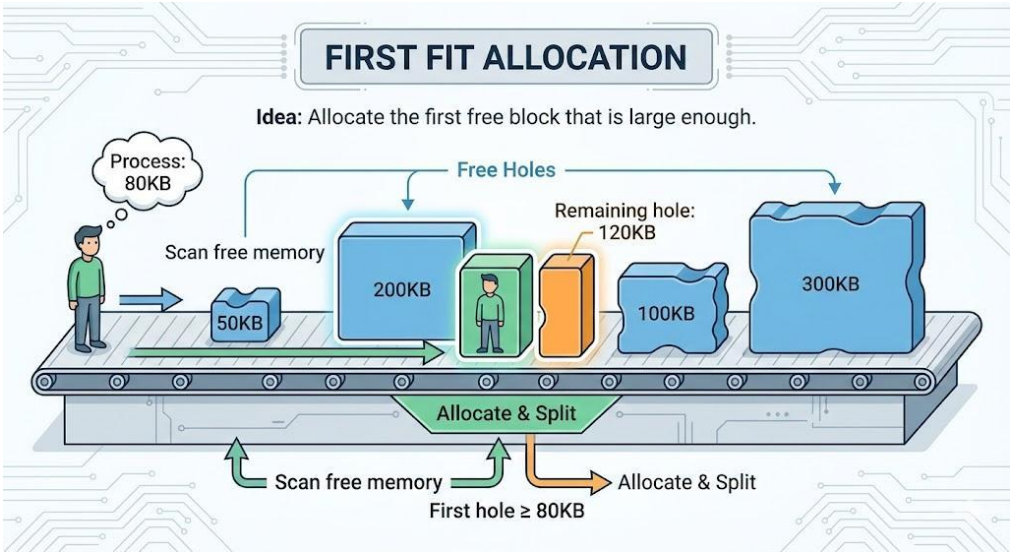
First Fit Allocation Algorithm

Newton School of Technology

🧠 **Idea**
Allocate the **first free block** that is **large enough** for the process.

⚙️ **How It Works**
1️⃣ Scan free memory **from the start**
2️⃣ Pick the **first hole ≥ process size**
3️⃣ Allocate memory
4️⃣ Split hole if needed
5️⃣ Stop searching

📌 **Example**
Free Holes: 50KB | 200KB | 100KB | 300KB
Process: 80KB
➡️ **Chosen:** 200KB (first fit)
➡️ **Remaining hole:** 120KB



First Fit Allocation Algorithm

Newton School of Technology

✅ **Pros**

- Fast & simple
- Low overhead
- Works well in practice

❌ **Cons**

- Small holes accumulate at start
- Not always optimal
- Fragmentation near beginning

⏱️ **Performance**

- **Time:** `O(n)` (scan free list)

Key Takeaway

First Fit is quick and simple, but may cause fragmentation over time.

First Fit Allocation Algorithm

Newton School of Technology

Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies
Block 1 (20K)	Yes	No	
Block 2 (100K)	No		
Block 3 (40K)	No		
Block 4 (200K)	No		
Block 5 (10K)	Yes	No	

None of the available blocks
Can accommodate P4. It remains unallocated

Practice Problem 1

Newton School of Technology

A system uses **fixed partition memory management** with the following partitions: **100KB, 500KB, 200KB, 300KB, 600KB**

The following processes arrive in order: **P1 = 212KB, P2 = 417KB, P3 = 112KB, P4 = 426KB.**

Allocate the processes using the **First Fit** algorithm. Show clearly which partition each process is allocated to. Identify any process that cannot be allocated. Calculate the **total internal fragmentation**.

Allocation (First Fit):

- **P1 (212KB)** → 500KB partition → waste = 288KB
- **P2 (417KB)** → 600KB partition → waste = 183KB
- **P3 (112KB)** → 200KB partition → waste = 88KB
- **P4 (426KB)** → ❌ no partition fits → **waiting**

Answers:

- **Allocated:** P1, P2, P3
- **Waiting:** P4 (1 process)
- **Total Internal Fragmentation:** 288 + 183 + 88 = **559KB**

Explanation: In fixed partitioning, process must fit entirely in one partition. Unused space inside partitions causes **internal fragmentation**.

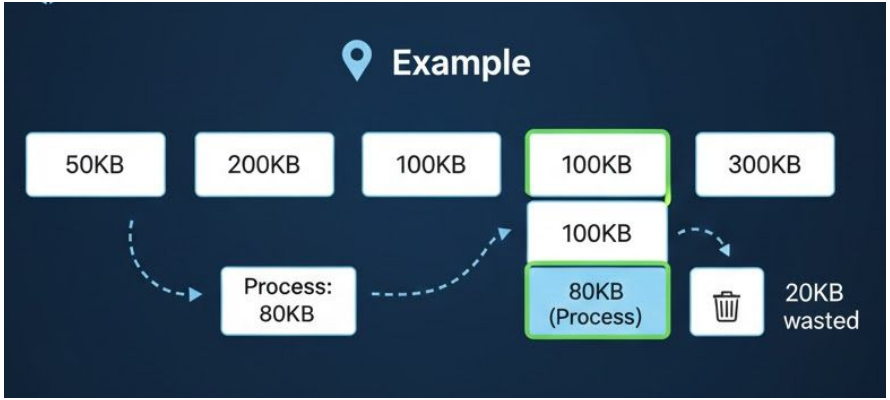
Best Fit Allocation Algorithm

Newton School of Technology

🧠 **Idea**
Allocate the **smallest free block** that can fit the process
➡️ Leaves **minimum leftover space**

⚙️ **How It Works**
1️⃣ Scan **entire free list**
2️⃣ Choose **smallest hole ≥ process size**
3️⃣ Allocate & split remainder

📌 **Example**
Free Holes: 50KB | 200KB | 100KB | 300KB
Process: 80KB
➡️ **Chosen:** 100KB (waste = 20KB)



Best Fit Allocation Algorithm

Newton School of Technology

✅ **Pros**

- Minimizes waste per allocation
- Saves large holes for big processes
- Good utilization (in theory)

❌ **Cons**

- Slower (full scan)
- Creates many tiny holes
- Often worse than First Fit in practice

⏱️ **Performance**

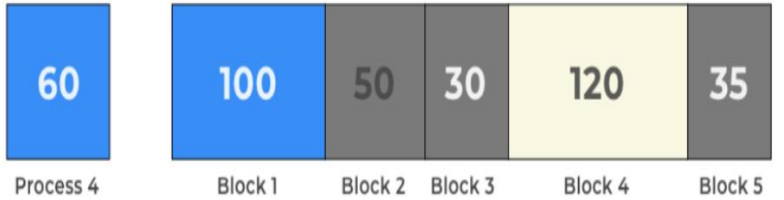
- **Time:** `O(n)` (checks all holes)

Key Takeaway

Best Fit reduces immediate waste but increases fragmentation and overhead over time.

Best Fit Allocation Algorithm

Newton School of Technology



Block / Size	Available	Can Occupy P3?	Memory Wastage After Process Occupies	
Block 1 (100K)	Yes	Yes	100 - 60 = 40	Best
Block 2 (50K)	No	-	-	
Block 3 (30K)	Yes	-	-	
Block 4 (120K)	Yes	Yes	120 - 60 = 60	
Block 5 (35K)	Yes	-	-	

Block 1 results in the least memory wastage

P4 goes to Block 1

Worst Fit Allocation Algorithm

Newton School of Technology

🧠 **Idea**

Allocate the **largest available hole**

➡️ Leave a **big leftover** that might still be useful

⚙️ **How It Works**

1️⃣ Scan **entire free list**

2️⃣ Pick **largest hole ≥ process size**

3️⃣ Allocate & keep large remainder

📌 **Example**

Free Holes: 15KB | 25KB | 40KB | 50KB

Process: 30KB

➡️ **Chosen:** 100KB

➡️ **Remaining:** 70KB

Worst Fit Allocation Algorithm

Newton School of Technology

✅ **Pros**

- Leftover space stays large
- Fewer tiny holes (initially)

❌ **Cons**

- Slow (full scan)
- Breaks large holes quickly
- Often **worst performer**

⏱️ **Performance**

- **Time:** `O(n)`

Key Takeaway

Worst Fit tries to preserve usable space, but usually causes poor overall memory utilization.

Worst Fit Allocation Algorithm

Newton School of Technology

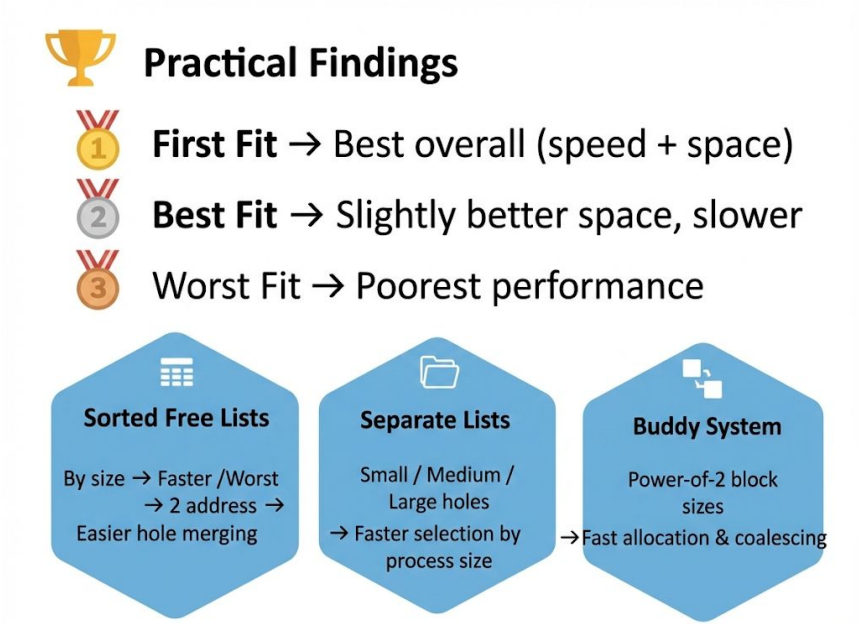
	Available	Can Occupy P4?	Memory Wastage After Process Occupies
Block 1	No		
Block 2	No		
Block 3	Yes	No	P4 remains unallocated, As can not be fit by any Free blocks
Block 4	No		
Block 5	Yes	No	

Comparison of Allocation Algorithms

Newton School of Technology

Practical Findings

- 🥇 **First Fit** → Best overall (speed + space)
- 🥈 **Best Fit** → Slightly better space, slower
- 🥉 **Worst Fit** → Poorest performance



✅ **Recommendation**

First Fit is preferred in practice: **simple, fast, and efficient**.

Practice Problem 2

Newton School of Technology

A system uses **fixed partition memory management** with the following partitions: **100KB | 200KB | 50KB | 300KB**

The processes arrive in the following order: **P1 = 70KB, P2 = 110KB, P3 = 40KB**

1. Allocate the processes using: (a) **First Fit** (b) **Best Fit** (c) **Worst Fit**
2. Show clearly which partition each process is allocated to under each algorithm.
3. Identify any unallocated process (if any).
4. Calculate the **total internal fragmentation** for each algorithm.

✅ **Best Fit**

- P1 (70) → 100 → **30KB internal**
- P2 (110) → 200 → **90KB internal**
- P3 (40) → 50 → **10KB internal**

Total Internal Fragmentation = 130KB
Unused partition: 300KB

✅ **First Fit**

- P1 (70) → 100 → **30KB internal**
- P2 (110) → 200 → **90KB internal**
- P3 (40) → 50 → **10KB internal**

Total Internal Fragmentation = 30 + 90 + 10 = 130KB
Unused partition: 300KB

✅ **Worst Fit**

- P1 (70) → 300 → **230KB internal**
- P2 (110) → 200 → **90KB internal**
- P3 (40) → 100 → **60KB internal**

Total Internal Fragmentation = 230 + 90 + 60 = 380KB
Unused partition: 50KB

Explanation:

- **First Fit:** Fast, practical
- **Best Fit:** Less waste per allocation, many tiny holes
- **Worst Fit:** Leaves big holes but performs worst overall

Memory Fragmentation

Newton School of Technology

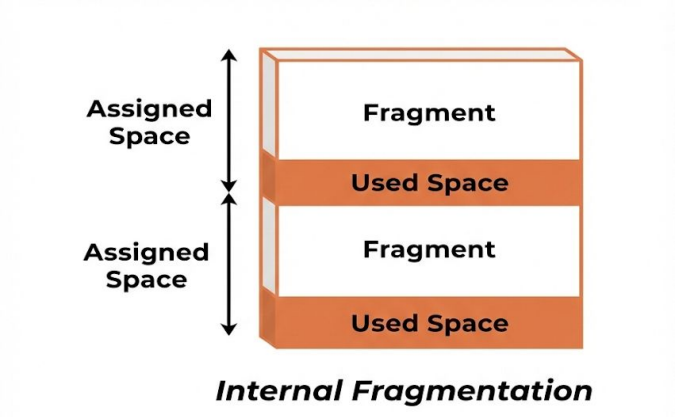
🧠 **Definition**
Fragmentation = **wasted memory** that cannot be effectively used.

🔹 **1. Internal Fragmentation**
- Waste **inside allocated memory**
- Happens in **Fixed Partitioning**
- Allocated > Needed

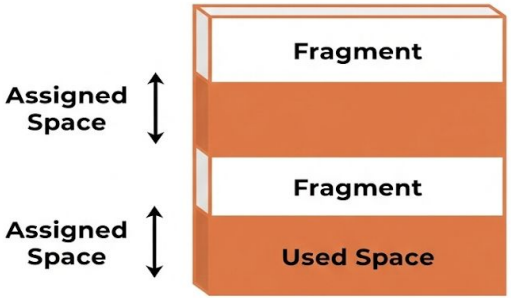
Example:
100KB partition − 70KB process = **30KB wasted**
Formula:
```
Internal = Partition Size - Process Size
```

🔸 **2. External Fragmentation**
- Waste **between processes**
- Happens in **Dynamic Partitioning**
- Free memory exists but **not contiguous**

Key Problem:
Enough total free memory, but **no single large block**



Internal Fragmentation



External Fragmentation

Solutions to Fragmentation – Compaction

Newton School of Technology

🧠 **What It Is**

Rearrange memory so **all free space becomes one large block**.

🔄 **How It Works**

- Move all processes to one end
- Merge scattered holes
- Update process addresses

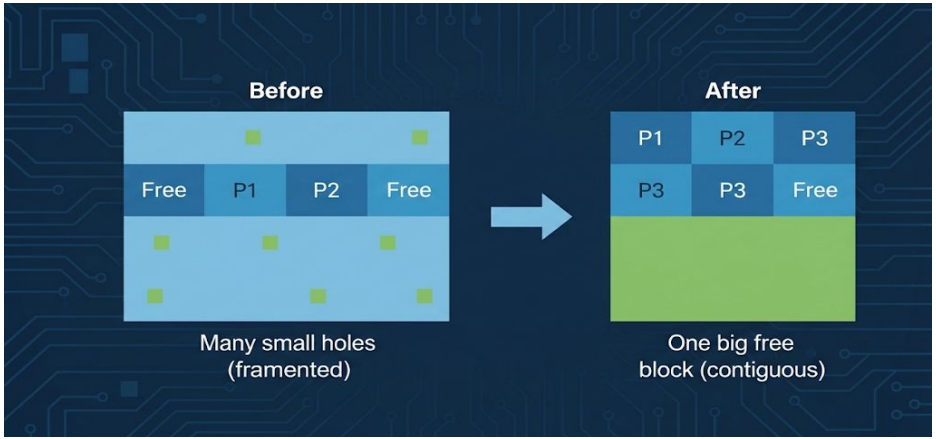
📊 **Effect**

- **Before:** Many small holes (fragmented)
- **After:** One big free block (contiguous)

⚙️ **Requirements**

- Dynamic relocation
- Hardware support (MMU)

Processes movable at runtime



Solutions to Fragmentation – Compaction

Newton School of Technology

✅ **Pros**

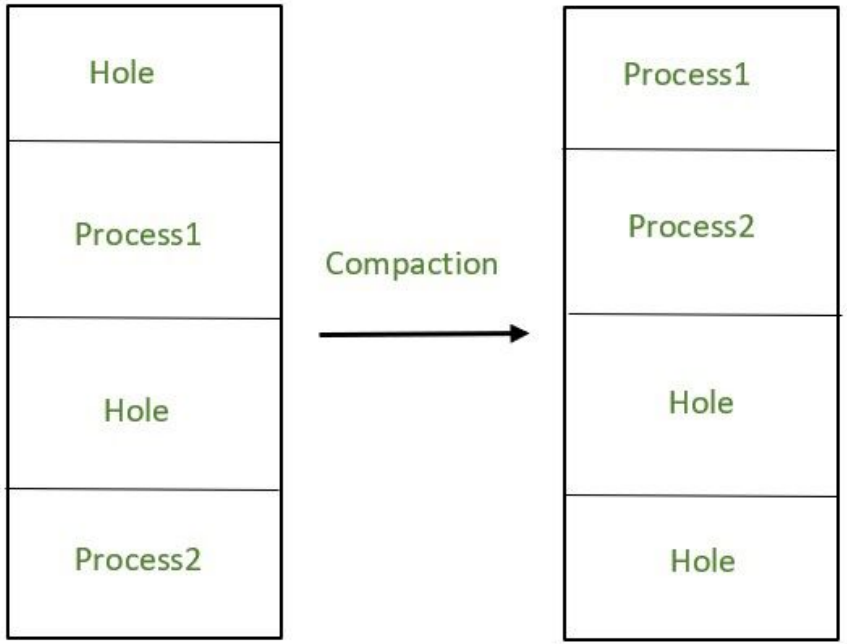
- Eliminates **external fragmentation**
- Improves memory utilization
- Enables large allocations

❌ **Cons**

- **Very expensive** (memory copying)
- System pause required
- Address updates needed
- Time ≈ **O(memory size)**

⏱️ **When to Use**

- Allocation fails despite enough total free memory
- Low system activity
- Fragmentation crosses threshold



Key Takeaway

Compaction fixes fragmentation but is costly—used sparingly in real systems.

Solutions to Fragmentation – Coalescing

Newton School of Technology

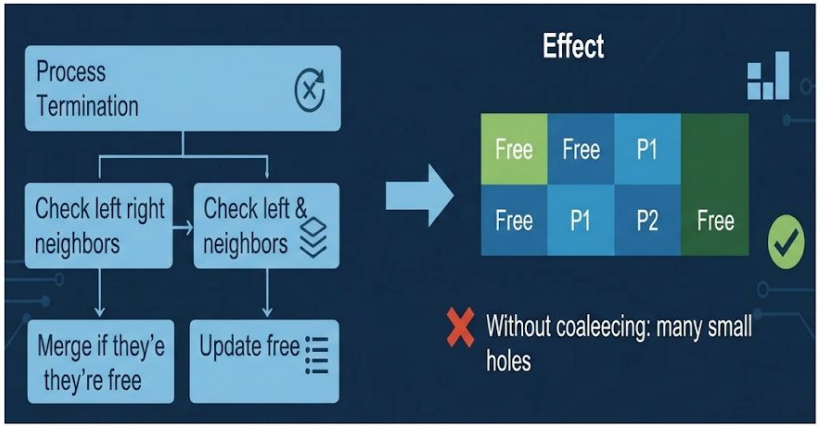
🧠 **What Is Coalescing?**
Automatically **merge adjacent free blocks** when memory is released.

🔄 **How It Works**

- On process termination:
  - Mark block as free
  - Check **left & right neighbors**
  - **Merge** if they're free
- Update free list

📊 **Effect**

- ❌ Without coalescing: many small holes
- ✅ With coalescing: **one larger hole**



Solutions to Fragmentation – Coalescing

Newton School of Technology

⚙️ **Implementation**

- 🏷️ **Boundary Tags**
  - Size/status at both ends
  - **O(1)** neighbor checks
- 🔗 **Doubly Linked Free List**
  - Easy merge with neighbors

✅ **Pros**

- Prevents fragmentation from worsening
- Fast (**O(1)**)
- No process relocation
- Automatic

❌ **Cons**

- Doesn’t remove existing fragmentation
- Extra bookkeeping

Key Takeaway

Coalescing keeps free memory blocks large by merging neighbors—cheap, fast, and essential for dynamic allocation.

Practice Problem 3

Newton School of Technology

A system has a total main memory of **1024 MB**. The operating system occupies **124 MB** of memory. Four processes are currently loaded into memory: P1 = 200 MB, P2 = 150 MB, P3 = 100 MB, P4 = 180 MB

After allocation, the remaining free memory is scattered into the following holes: **60 MB, 40 MB, 90 MB, and 80 MB**

```
| OS 124 | P1 200 | Hole 60 | P2 150 | Hole 40 |
| P3 100 | Hole 90 | P4 180 | Hole 80 |
```

1. Calculate the **total memory and free memory.**.
2. Compute the **memory utilization percentage**.
3. Determine the **total external fragmentation**.
4. Identify the **largest available hole**.
5. Determine whether a new process of **120 MB** can be allocated. Justify your answer. Is this possible after compaction?

Calculate Used Memory
Processes: 200+150+100+180=630 MB. Total Used: 124+630=754 MB
Free Memory: 1024−754=270 MB free

🎨 **Scattered Holes**
Holes: 60 MB, 40 MB, 90 MB, 80 MB
Total free: 60+40+90+80=270 MB

📊 **(a) Memory Utilization**
754/1024×100≈73.6%
✔ Utilization ≈ **74%**

📊 **(b) External Fragmentation**
Total Free = **270 MB**
Largest Hole = **90 MB**
Memory is scattered into 4 separate holes → **External Fragmentation exists**

(c) Can a 120MB Process Be Allocated?
Required = 120 MB
Largest hole = 90 MB
Even though total free = 270 MB, ❌ Cannot allocate (no contiguous 120 MB block)

🧩 **After Compaction (if OS Rearranges Memory)**

```
| OS | P1 | P2 | P3 | P4 | Free 270 |
```

Now: ✔ 120MB process can be allocated
Remaining free = 150 MB

Swapping in Memory Management

Newton School of Technology

🧠 **What is Swapping?**
Moving **entire processes** between **RAM** and **disk** to free memory.

🎯 **Why Swapping?**
- Increase **multiprogramming**
- Free memory for **high-priority** processes

🔄 **How It Works**
- **Swap Out:** RAM → Disk
- **Swap In:** Disk → RAM

⏱️ **Cost**
- Very **expensive** (disk I/O)
- Much slower than context switch

Swap Time
`= Process Size / Disk Transfer Rate`
Example: 100MB ÷ 50MB/s → **2s out + 2s in = 4s**

⚠️ **Key Points**
- Needs backing store (disk)
- Requires contiguous memory (with swapping)
- Excessive swapping → **thrashing**

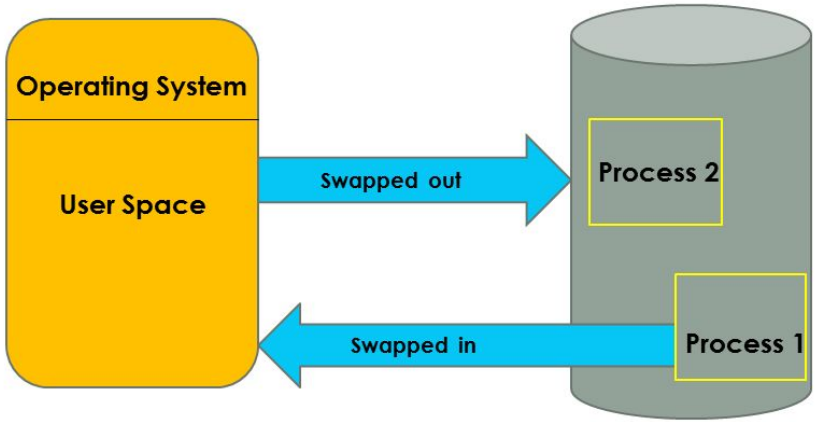
Key Takeaway
Swapping extends memory capacity but is costly—used carefully in real systems.

Implementing Swapping

Newton School of Technology

⚙️ **Core Requirements**

- 💽 **Backing Store:** Fast disk space for swapped processes
- 🔁 **Dynamic Relocation:** Process may return to different memory location
- 🔄 **Roll Out / Roll In:**
  - Low priority → swapped out
  - High priority → swapped in



⏱️ **What Affects Swap Time**

- 📦 **Process Size** → larger = slower
- 💾 **Disk Speed** → SSD ≫ HDD
- 🧠 **Memory Load** → may need extra swaps

🧠 **Swapping Policies**

When to Swap Out

- Low free memory, Long I/O wait and Lower priority process

Which Process

- Lowest priority, Longest blocked and Largest memory user

Overlay Technique

Newton School of Technology

🧠 **What Is It?**

Run programs **larger than RAM** by loading **only needed code/data** at a time.

📜 **Background**

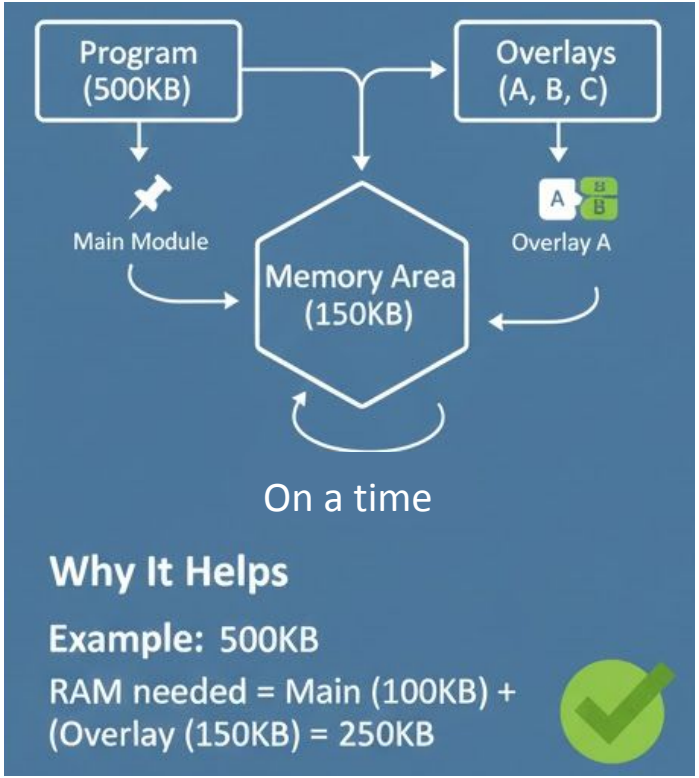
- Used **before virtual memory**
- **Manual** (programmer-managed)
- Common in **early / low-memory systems**

⚙️ **How It Works**

- Program split into **main module + overlays**
- **Main module** stays in memory
- **One overlay at a time** loaded on demand
- Overlays **replace each other** in the same memory area

📊 **Why It Helps**

- Example:
  - Program = **500KB**
  - RAM needed = **Main (100KB) + Largest Overlay (150KB) = 250KB**



Overlay Technique

✅ **Pros**

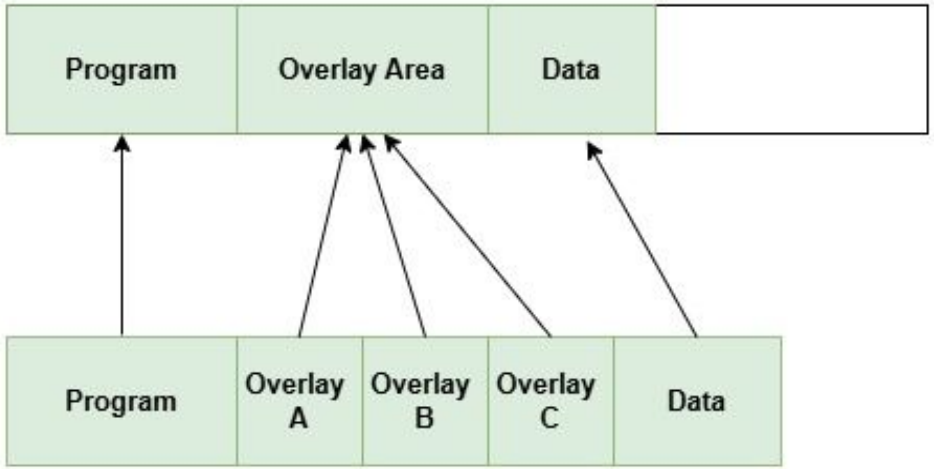
- Run large programs on small memory
- No special hardware
- Predictable memory usage

❌ **Cons**

- Complex & error-prone
- Programmer burden
- Hard to maintain

⚖️ **Overlay vs Virtual Memory**

- **Overlay:** Manual, programmer-controlled
- **Virtual Memory:** Automatic, OS-managed



Key Takeaway

Overlays were an early workaround for limited memory—powerful but painful compared to modern virtual memory.

Partitioning – Pros and Cons

Newton School of Technology

🧱 Fixed Partitioning

👍 Pros

- Simple & predictable
- Low management overhead
- Easy protection
- No external fragmentation

👎 Cons

- Internal fragmentation (waste)
- Fixed number of processes
- Inflexible sizes
- Large jobs may not fit

🔁 Dynamic Partitioning

👍 Pros

- No internal fragmentation
- Flexible sizes
- Better initial utilization
- Fits varied processes

👎 Cons

- External fragmentation
- Complex allocation
- Compaction is costly
- Variable allocation time

Contiguous Allocation – Pros and Cons

Newton School of Technology

⚙️ **Overall (Contiguous Allocation)**

👍 **Strengths**

- Fast access
- Simple address translation
- Minimal hardware
- Good locality

👎 **Limitations**

- Fragmentation issues
- Whole process must fit
- Limited sharing
- Size bound by largest hole

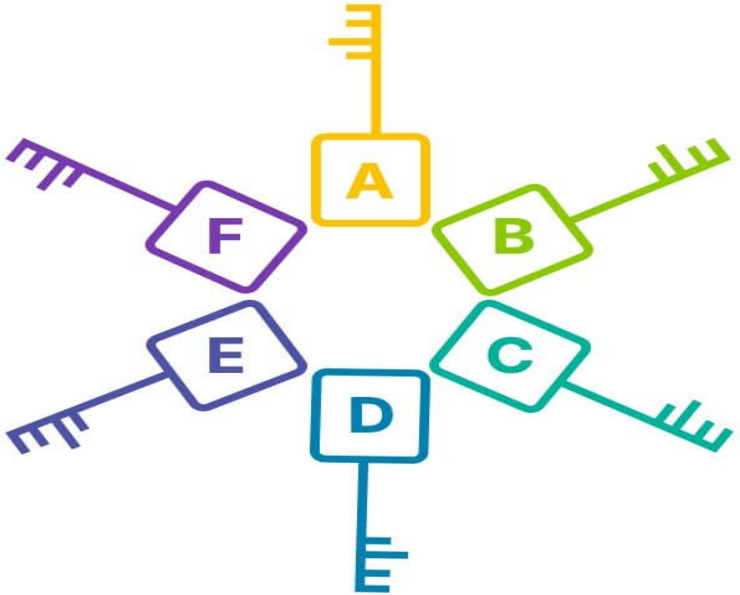
🚀 **Modern Alternatives**

- **Paging:** Removes external fragmentation
- **Segmentation:** Logical program view
- **Virtual Memory:** Beyond physical limits
- **Hybrid:** Paging + Segmentation

Key Takeaway:
Contiguous allocation is fast and simple—but fragmentation limits scalability in modern systems.

Key Takeaways

- Memory Hierarchy
- Contiguous Allocation
- Protection
- Fixed Partitioning
- Dynamic Partitioning
- Allocation Algorithms
- Fragmentation
- Swapping
- Overlays



Thank You!

Newton School of Technology

Thank you for your attention. Feel free to ask questions or revisit the topics covered.